P2266R0
Simpler implicit move

Published Proposal, 2021-01-06

Author:
Arthur O'Dwyer

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Draft Revision:
7

Current Source:
github.com/Quuxplusone/draft/blob/gh-pages/d2266-implicit-move-rvalue-ref.bs

Current:
rawgit.com/Quuxplusone/draft/gh-pages/d2266-implicit-move-rvalue-ref.html

---

Abstract

In C++20, return statements can *implicitly move* from local variables of rvalue reference type; but a defect in the wording means that *implicit move* fails to apply to functions that return references. C++20's implicit move is specified via a complicated process involving two overload resolutions, which is hard to implement, causing implementation divergence. We fix the defect and simplify the spec by saying that a returned move-eligible id-expression is *always* an xvalue.

Table of Contents

§ 1. Changelog

- None yet.

§ 2. Background

Starting in C++11, *implicit move* ([class.copy.elision]/3) permits us to return move-only types by value:

```
struct Widget {
    Widget(Widget&&);
};

Widget one(Widget w) {
    return w;  // OK since C++11
}
```

This wording was amended by [CWG1579], which made it legal to call converting constructors accepting an rvalue reference of the returned expression's type.

```
struct RRefTaker {
    RRefTaker(Widget&&);
};
RRefTaker two(Widget w) {
    return w;  // OK since C++11 + CWG1579
}
```

C++20 adopted [P1825], a wording paper created by merging [P0527] and [P1155]. The former introduced the category of "implicitly movable entities," and extended that category to include automatic variables of rvalue reference type. The latter increased the scope of the "implicit move" optimization beyond converting constructors — now, in C++20, the rule is simply that the first overload resolution to initialize the returned object is done by treating `w` as an rvalue. (The resolution may now produce candidates such as conversion operators and constructors-taking-`Base&&`.) Of these two changes, P0527's was the more drastic:

```
RRefTaker three(Widget&& w) {
    return w;  // OK since C++20 because P0527
}
```

However, due to the placement of P1825's new wording in [class.copy.elision]/3, the new wording about "implicitly movable entities" is triggered *only when initializing a return object.* Functions that do not return objects, do not benefit from this wording. This leads to a surprising result:

```
Widget&& four(Widget&& w) {
    return w;  // Error
}
```

In `return w`, the implicitly movable entity `w` is treated as an rvalue when the return type of the function is `RRefTaker` as in example `three`, but it is treated as an lvalue when the return type of the function is `Widget&&` as in example `four`.

§ 3. Problems remaining in C++20

§ 3.1. Conversion operators are treated inconsistently

```
struct Mutt {
    operator int*() &&;
};
struct Jeff {
    operator int&() &&;
};

int* five(Mutt x) {
    return x;  // OK since C++20 because P1155
}

int& six(Jeff x) {
    return x;  // Error
}
```

(`Mutt` here is isomorphic to example `nine` from [P1155]. P1155 did not explicitly consider `Jeff` because, at the time, Arthur hadn't realized that the difference between `Mutt` and `Jeff` was significant to the wording.)

§ 3.2. "Perfect backwarding" is treated inconsistently

```
template<class T>
T&& seven(T&& x) { return x; }

void test_seven(Widget w) {
    Widget& r = seven(w);                // OK
    Widget&& rr = seven(std::move(w));  // Error
}
```

The line marked "Error" instantiates `seven<Widget>`, with the signature `Widget&& seven(Widget&& x)`. The rvalue-reference parameter `x` is an implicitly movable entity according to C++20; but, because the return type is not an object type, implicit move fails to happen — the return type `Widget&&` cannot bind to the lvalue *id-expression* `x`.

The same surprise occurs with `decltype(auto)` return types:

```
Widget val();
Widget& lref();
Widget&& rref();

decltype(auto) eight() {
    decltype(auto) x = val();  // OK, x is Widget
    return x;  // OK, return type is Widget, we get copy elision
}

decltype(auto) nine() {
    decltype(auto) x = lref();  // OK, x is Widget&
    return x;  // OK, return type is Widget&
}

decltype(auto) ten() {
    decltype(auto) x = rref();  // OK, x is Widget&&
    return x;  // Error, return type is Widget&&, cannot bind to x
}
```

We propose to make `ten` work, by permitting — in fact *requiring* —the move-eligible *id-expression* `x` to be treated as an rvalue.

§ 3.2.1. Interaction with `decltype` and `decltype(auto)`

We do not propose to change any of the rules around the deduction of `decltype(auto)` itself. However, functions with `decltype(auto)` return types have some subtlety to them.

Consider this extremely contrived example:

```
decltype(auto) eleven(Widget&& x) {
    return (x);
}
```

Here, the return type of `eleven` is the decltype of the expression `(x)`. This is governed by [dcl.type.auto.deduct]/5:

> If the *placeholder-type-specifier* is of the form *type-constraint*$_{opt}$ `decltype(auto)`, `T` shall be the placeholder alone. The type deduced for `T` is determined as described in [dcl.type.decltype], as though *E* had been the operand of the decltype.

In C++17, the decltype of `(x)` was `int&`. No implicit move happened, because `x` (being a reference) was not an implicitly movable entity. The lvalue expression `(x)` happily binds to the function return type `int&`, and the code compiles OK.

In C++20, the decltype of `(x)` is `int&`. `x` now *is* an implicitly movable entity, but (because the return type is not an object type) implicit move does not apply. The lvalue expression `(x)` happily binds to the function return type `int&`, and the code compiles OK.

We propose to change the behavior of `eleven`!

Under our proposal, the *id-expression* `x` (as the operand of `return`) is *move-eligible*, which means it is an xvalue. The function return type is deduced as `decltype(`*E*`)`, which is to say, `int&&` since *E* is an xvalue. The xvalue expression `(x)` happily binds to the function return type `int&&`, and the code compiles OK. **But now it returns `int&&`, not `int&`.**

This does produce surprising inconsistencies in the handling of parentheses; for example,

```
auto f1(int x) -> decltype(x) { return (x); }     // int
auto f2(int x) -> decltype((x)) { return (x); }   // int&
auto f3(int x) -> decltype(auto) { return (x); }  // C++20: int&. Proposed:
auto g1(int x) -> decltype(x) { return x; }       // int
auto g2(int x) -> decltype((x)) { return x; }     // int&
auto g3(int x) -> decltype(auto) { return x; }    // int
```

Note that `f2` and `g2` are well-formed in C++20, but we propose to make `f2` and `g2` ill-formed, because they attempt to bind an lvalue reference to a *move-eligible* xvalue expression.

However, C++ users already know to be wary of parentheses anywhere in the vicinity of `decltype` or `decltype(auto)`. We don't think we're adding any significant amount of surprise in this already-arcane area.

§ 3.3. Two overload resolutions are overly confusing

Implicit move is currently expressed in terms of two separate overload resolutions: one treating the operand as an rvalue, and then (if that resolution fails) another one treating the operand as an lvalue.

As far as I know, this is the only place in the language where two separate resolutions are done on the same operand. This mechanism has some counterintuitive ramifications —not *problems* per se, but surprising and subtle quirks that would be nice to simplify out of the language.

```
struct Sam {
    Sam(Widget&);        // #1
    Sam(const Widget&);  // #2
};

Sam twelve() {
    Widget w;
    return w;  // calls #2 since C++20 because P1155
}
```

> Note: In C++17 (prior to P1155), #2 would not be found by the first pass because its argument type is not exactly `Widget&&`. The comment in `twelve` matches the current Standard wording, and matches the behavior of MSVC 19.27 and GCC 7 through 10. (As of this writing, GCC trunk has regressed and lost the correct behavior.) Clang 11 still calls #1 in all modes, because it has not yet implemented P1155 (but Clang will correctly call #2 after [D88220] is landed).

The first overload resolution succeeds, and selects a candidate (#2) that is a *worse match* than the candidate that would have been selected by the second overload resolution. This is a surprising quirk, which was discussed internally around the time P1825 was adopted (see [CoreReflector]); that discussion petered out with no conclusion except a general sense that the alternative mechanisms discussed (such as introducing a notion of "lvalues that preferentially bind to rvalue references" or "rvalues that reluctantly bind to lvalue references") were strictly worse than the status quo.

```
struct Frodo {
    Frodo(Widget&);
    Frodo(Widget&&) = delete;
};

Frodo thirteen() {
    Widget w;
    return w;  // Error: the first overload resolution selects a deleted fu
}
```

Here the first pass uniquely finds `Frodo(Widget&&)`, which is a deleted function; does this count as "the first overload resolution fails," or does it count as a success and thus produce an error when we try

to use that deleted function? Vendors currently disagree, but [over.match.general]/3 is clear:

> If a best viable function exists and is unique, overload resolution succeeds and produces it as the result. Otherwise overload resolution fails and the invocation is ill-formed. [...] Overload resolution results in a *usable candidate* if overload resolution succeeds and the selected candidate is either not a function ([over.built]), or is a function that is not deleted and is accessible from the context in which overload resolution was performed.

Error from use of deleted function: GCC 5,6,7; GCC trunk with `-std=c++20` ; MSVC; ICC

Non-conforming fallback to `Frodo(Widget&)`: GCC 8,9,10; GCC trunk with `-std=c++17` ; Clang before [D92936]

This implementation divergence would be less likely to exist, if the specification were simplified to avoid relying on the precise formal meaning of "failure." We propose that simplification.

Another example of vendors misinterpreting the meaning of "failure":

```
struct Merry {};
struct Pippin {};
struct Together : Merry, Pippin {};
struct Quest {
    Quest(Merry&&);
    Quest(Pippin&&);
    Quest(Together&);
};

Quest fourteen() {
    Together t;
    return t;  // C++20: calls Quest(Together&). Proposed: ill-formed
}
```

Here the first pass finds both `Quest(Merry&&)` and `Quest(Pippin&&)`. [over.match.general]/3 is clear that ambiguity *is* an overload resolution failure and the second resolution must be performed. However, EDG's front-end disagrees.

Fallback to `Quest(Together&)`: GCC; Clang; MSVC

Non-conforming error due to ambiguity in the first pass: ICC

§ 3.4. A specific case involving `reference_wrapper`

Consider this dangerous function:

```
std::reference_wrapper<Widget> fifteen() {
    Widget w;
    return w;  // OK until CWG1579; OK after LWG2993. Proposed: ill-formed
}
```

Prior to [CWG1579] (circa 2014), implicit move was not done, and so `w` was treated as an lvalue and `fifteen` was well-formed — it returned a dangling reference to automatic variable `w`.

CWG1579 made `fifteen` ill-formed (except on the non-conforming compilers listed above), because now the first overload resolution step would find `reference_wrapper(type&&) = delete` and hard-error.

Then, [LWG2993] eliminated this deleted constructor from `reference_wrapper` and replaced it with a SFINAE-constrained constructor from `U&&`. Now, the first overload resolution step legitimately fails (it finds no viable candidates), and so the second overload resolution is performed and finds a usable candidate — it returns a dangling reference to automatic variable `w`. This is how the situation stands today in C++20.

We propose to simplify [class.copy.elision]/3 by eliminating the second "fallback" overload resolution. If this proposal is adopted, `fifteen` will once again become ill-formed.

In the internal discussion of P1825 ([CoreReflector]) one participant opined that making `fifteen` ill-formed is a good thing, because it correctly diagnoses the dangling reference. The existing two-step mechanism works to defeat the clear intent of `reference_wrapper`'s SFINAE-constrained constructor and permit the returning of dangling references when in fact we don't want that.

§ 4. Proposed wording relative to C++20

Note that this proposal merge-conflicts with Jens Maurer's [DraftPR3998]. Consensus seems to be that [class.copy.elision] is no longer the best place to explain "implicit move." [DraftPR3998] moves the "implicitly movable entity" wording from [class.copy.elision] to [dcl.init], and introduces the term "result-initialization" for copy-initialization contexts in which implicit move can happen. Vice versa, we propose to move the wording from [class.copy.elision] to [expr.prim.id.unqual], and introduce the term "move-eligible *id-expression*" for xvalue id-expressions.

Modify [expr.prim.id.unqual]/2 as follows:

> The expression is an xvalue if it is move-eligible (see below); an lvalue if the entity is a function, variable, structured binding, data member, or template parameter object ; and a prvalue otherwise; it is a bit-field if the identifier designates a bit-field.
>
> An *implicitly movable entity* is a variable of automatic storage duration that is either a non-volatile object or an rvalue reference to a non-volatile object type. In the following contexts, an *id-expression* is *move-eligible*:
>
> - If the *id-expression* (possibly parenthesized) is the operand of a `return` or `co_return` statement, and names an implicitly movable entity declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression*, or
> - if the *id-expression* (possibly parenthesized) is the operand of a *throw-expression*, and names an implicitly movable entity whose scope does not extend beyond the *compound-statement* of the innermost *try-block* or *function-try-block* (if any) whose *compound-statement* or *ctor-initializer* encloses the *throw-expression*.

Eliminate [class.copy.elision]/3:

> ~~An *implicitly movable entity* is a variable of automatic storage duration that is either a non-volatile object or an rvalue reference to a non-volatile object type. In the following copy-initialization contexts, a move operation is first considered before attempting a copy operation:~~
>
> - ~~If the *expression* in a `return` or `co_return` statement is a (possibly parenthesized) *id-expression* that names an implicitly movable entity declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression*, or~~
> - ~~if the operand of a *throw-expression* is a (possibly parenthesized) *id-expression* that names an implicitly movable entity whose scope does not extend beyond the *compound-statement* of the innermost *try-block* or *function-try-block* (if any) whose *compound-statement* or *ctor-initializer* encloses the *throw-expression*,~~
>
> ~~overload resolution to select the constructor for the copy or the `return_value` overload to call is first performed as if the expression or operand were an rvalue. If the first overload resolution fails or was not performed, overload resolution is performed again, considering the expression or operand as an lvalue.~~ ~~[*Note 3:* This two-stage overload resolution is performed regardless of whether copy elision will occur. It determines the constructor or the `return_value` overload to be called if elision is not performed, and the selected constructor or `return_value` overload must be accessible even if the call is elided. — *end note*]~~

Also change the definition of `g()` in [class.copy.elision]/4:

```
struct Weird {
  Weird();
  Weird(Weird&);
};

Weird g() {
  Weird w;
  return w;  // OK: first overload resolution fails, second overload resolution selects
}
Weird g(bool b) {
  static Weird w1;
  Weird w2;
  if (b) {
    return w1;  // OK: Weird(Weird&)
  } else {
    return w2;  // error: w2 in this context is an xvalue
  }
}
```

§ 4.1. Non-normative clarifications

Modify [dcl.type.auto.deduct]/5 as follows:

> If the *placeholder-type-specifier* is of the form *type-constraint*$_{opt}$ `decltype(auto)`, `T` shall be the placeholder alone. The type deduced for `T` is determined as described in [dcl.type.decltype], as though *E* had been the operand of the `decltype`. [*Example:*
>
> ```
> auto f(int x) -> decltype((x)) { return (x); }  // return type is "int&"
> auto g(int x) -> decltype(auto) { return (x); }  // return type is "int&&"
> ```
>
> *— end example*]

Add yet more examples to [class.copy.elision]/4, showing how the new wording affects functions that return references:

```
int& h(bool b, int i) {
  static int s;
  if (b) {
    return s;  // OK
  } else {
    return i;  // error: i is an xvalue
  }
}

decltype(auto) h2(Thing t) {
  return t;  // OK: t is an xvalue and h2's return type is Thing
}

decltype(auto) h3(Thing t) {
  return (t);  // OK: (t) is an xvalue and h3's return type is Thing&&
}
```

Add a note after [dcl.init.ref]/5.4.4:

> if the reference is an rvalue reference, the initializer expression shall not be an lvalue.
>
> [*Note:* This can be affected by whether the initializer expression is move-eligible ([expr.prim.id.unqual]). — *end note*]

§ 5. Acknowledgments

- Thanks to Ville Voutilainen for recommending Arthur write this paper.
- Thanks to Aaron Puchert for inspiring `fourteen` via his comments on [D68845].
- Thanks to Richard Smith and Jens Maurer for their feedback, and to Jens for proposing the term "move-eligible."

§ References

§ Informative References

[CoreReflector]
CWG internal email discussion. [isocpp-core] P1825 (more implicit moves) surprise. February 2020. URL: https://lists.isocpp.org/core/2020/02/8455.php

[CWG1579]

Jeffrey Yasskin. Return by converting move constructor. October 2012. URL: http://open-std.org/JTC1/SC22/WG21/docs/cwg_defects.html#1579

[D68845]

Aaron Puchert. Don't emit unwanted constructor calls in co_return statements. October 2019. URL: https://reviews.llvm.org/D68845

[D88220]

Yang Fan. [C++20] P1825R0: More implicit moves. September 2020. URL: https://reviews.llvm.org/D88220

[D92936]

Yang Fan. [Sema] Fix deleted function problem in implicitly movable test. December 2020. URL: https://reviews.llvm.org/D92936

[DraftPR3998]

Jens Maurer. [class.copy.elision] Move specification of altered overload resolution. May 2020. URL: https://github.com/cplusplus/draft/pull/3998/

[LWG2993]

Tim Song. reference_wrapper<T> conversion from T&&. November 2017. URL: https://cplusplus.github.io/LWG/issue2993

[P0527]

David Stone. Implicitly move from rvalue references in return statements. November 2017. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0527r1.html

[P1155]

Arthur O'Dwyer; David Stone. More implicit moves. June 2019. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1155r3.html

[P1825]

David Stone. Merged wording for P0527R1 and P1155R3. July 2019. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1825r0.html